

Project 7

Project Title: Health Monitoring Dashboard

Objective

The objective of this project is to develop a web-based health monitoring dashboard that allows users to track their daily fitness activities such as steps taken, calories burned, and heart rate trends. The application will leverage APIs such as Fitbit API or Google Fit API to fetch real-time data and visualize the data using interactive graphs. The project will be developed using PHP and MySQL.

Scope of the Project

1. **User-centric Functionality:**
 - Display daily fitness metrics including steps taken, calories burned, and heart rate trends.
 - Visualize historical data using interactive charts to monitor progress over time.
 - Allow users to synchronize fitness data through API integration with Fitbit or Google Fit.
2. **Web-based Solution:**
 - The dashboard will be implemented as a responsive web application accessible across devices.
 - Provide a user-friendly interface with cards summarizing key metrics and an interactive graph section for trends.
3. **Technology Stack:**
 - **Frontend:** HTML, CSS (Bootstrap), JavaScript (Chart.js or D3.js).
 - **Backend:** PHP for server-side scripting.
 - **Database:** MySQL for data storage and management.
 - **APIs:** Fitbit API or Google Fit API for retrieving user fitness data.

Functional Requirements

1. **User Authentication:**
 - Registration and login functionality.
 - Secure authentication with password hashing.
2. **Daily Activity Stats:**
 - Display metrics for steps, calories burned, and heart rate.
 - Retrieve data from Fitbit or Google Fit API using OAuth2 authentication.
3. **Graphical Visualizations:**
 - Use interactive graphs (e.g., line, bar, and pie charts) to display trends over customizable time ranges (daily, weekly, monthly).
4. **Data Synchronization:**
 - Allow users to manually sync data with APIs.
 - Schedule automated background synchronization at regular intervals.
5. **Responsive Layout:**
 - Dashboard-style layout with cards for metrics and a dedicated graph section.
 - Ensure mobile and desktop compatibility.

Non-functional Requirements

1. **Performance:**
 - Load dashboard data and visualizations within 3 seconds.
 - Optimize API requests and database queries for efficiency.
2. **Scalability:**

- Support multiple users with concurrent access.
- 3. **Security:**
 - Secure API calls using HTTPS.
 - Encrypt sensitive user data in the database.
- 4. **Usability:**
 - Intuitive interface with a clean design and easy navigation.

System Architecture

1. **Frontend:**
 - HTML for structure, CSS (Bootstrap) for styling, and JavaScript for interactivity.
 - Use Chart.js or D3.js for creating dynamic and interactive graphs.
2. **Backend:**
 - PHP to handle user authentication, API requests, and database operations.
3. **Database:**
 - MySQL to store user data, daily metrics, and historical trends.
4. **API Integration:**
 - Fitbit or Google Fit API integration using OAuth2 for user authentication and data retrieval.

Use Case Scenarios

Use Case 1: User Registration and Login

- **Actors:** New User
- **Scenario:**
 1. User registers by providing a username, email, and password.
 2. Password is hashed and stored in the database.
 3. User logs in to access their dashboard.

Use Case 2: Connecting to Fitbit or Google Fit

- **Actors:** Registered User
- **Scenario:**
 1. User clicks "Connect to Fitbit/Google Fit".
 2. The system redirects the user to the API's OAuth2 authorization page.
 3. The user authorizes access to their fitness data.
 4. API access and refresh tokens are securely stored in the database.

Use Case 3: Viewing Daily Metrics

- **Actors:** Registered User
- **Scenario:**
 1. User logs in to the dashboard.
 2. Daily activity stats (steps, calories, heart rate) are retrieved from the database.
 3. Metrics are displayed in cards on the dashboard.

Use Case 4: Viewing Trends Over Time

- **Actors:** Registered User
- **Scenario:**
 1. User selects a time range (e.g., weekly or monthly).
 2. Historical data is fetched from the database.
 3. Interactive graphs display trends for the selected time range.

Use Case 5: Manual Data Synchronization

- **Actors:** Registered User
- **Scenario:**
 1. User clicks "Sync Data".
 2. The system fetches the latest data from the API.
 3. Data is updated in the database and reflected on the dashboard.

Implementation Steps

Step 1: Setup Development Environment

- Install PHP, MySQL, and Apache (XAMPP or LAMP stack).
- Set up a local development environment for testing.

Step 2: API Integration

- Register the application with Fitbit or Google Fit to obtain API credentials.
- Implement OAuth2 authentication for API access using PHP cURL.

Step 3: Backend Development

- Implement user authentication using PHP sessions.
- Write scripts for fetching and storing data from APIs into the database.

Step 4: Frontend Development

- Design the dashboard layout using Bootstrap.
- Implement graphs using Chart.js or D3.js.
- Use AJAX for dynamic content updates.

Step 5: Testing and Debugging

- Test all functionalities, including user authentication, API synchronization, and graph visualizations.
- Fix any bugs or performance issues.

Step 6: Deployment

- Deploy the application on a web server (e.g., AWS, DigitalOcean, or a university server).
- Secure the application with HTTPS using an SSL certificate.

Evaluation Metrics

1. **Functionality:** Ensure all features (e.g., API integration, graph visualizations) are working correctly.
2. **Performance:** Measure response times for API calls and data loading.
3. **Usability:** Evaluate the user interface for ease of navigation and interaction.
4. **Security:** Verify the use of secure practices (e.g., encrypted passwords, HTTPS).